Enterprise Multi-Cluster Architecture: Comprehensive Administration Control Plane Initialization

The architectural imperative of modern, highly available infrastructure dictates a strict separation of concerns between management tooling and application workloads. Within an enterprise-grade three-cluster topology—comprising Administration, Development, and Production environments—the Administration (Admin) cluster functions as the centralized orchestration and governance hub[1]. It provides global fleet management, unified Identity and Access Management (IAM), Continuous Integration and Continuous Delivery (CI/CD) pipelines, and a centralized telemetry database[1]. This isolation ensures that resource-intensive compilation processes, artifact registry indexing, or log aggregation queries do not starve consumer-facing production workloads of compute capacity[1].

The following exhaustive architectural report details the end-to-end initialization of the Admin cluster using Rancher Kubernetes Engine 2 (RKE2) on a single-node virtual machine architecture. The ultimate objective of this deployment is to establish a self-sufficient management plane capable of autonomously monitoring, securing, and deploying workloads to external Development and Production clusters across a bridged virtual network[1].

1. Architectural Blueprint and Logical Hierarchy

Before initializing the underlying orchestration engine, the logical and physical boundaries of the infrastructure must be established. The deployment environment utilizes bridged networking on a host machine, providing the Virtual Machines (VMs) with a shared, localized private subnet[1]. The Admin node operates on the static IP address 192.168.56.10, while the Development and Production nodes operate on 192.168.56.11 and 192.168.56.12, respectively[1]. The routing layer relies on the .nip.io wildcard DNS service, which seamlessly translates domain names directly to the Admin node's host-only interface[1]. This strategy eliminates the requirement for manual hosts file modifications across collaborative endpoints, allowing external development teams to resolve services like ArgoCD or Sonatype Nexus natively through the cluster's ingress controller[1].

1.1 The Namespace Segregation Strategy

To prevent resource conflicts and maintain granular Role-Based Access Control (RBAC), the Admin cluster is strictly partitioned into functional namespaces[1]. In Kubernetes, the default kube-system namespace is highly sensitive and reserved exclusively for core internal networking components such as CoreDNS and the Container Network Interface (CNI)[1]. Deploying heavy operational tools within this reserved space introduces severe routing risks. Therefore, the administrative toolchain is distributed across dedicated organizational partitions[1].

Namespace Name	Functional Purpose	Contained Services and Workloads
kube-system	Core cluster routing, internal networking, and DNS resolution.	RKE2 Core (etcd, apiserver, scheduler), Canal CNI, NGINX Ingress Controller[1].
cattle-system	Global cluster fleet management and governance.	Rancher Manager multi-cluster control plane[1].
identity-core	Central authentication and identity directory.	Keycloak Single Sign-On (SSO) engine, PostgreSQL backing database[1].
storage-system	S3-compatible persistent object storage backbone.	MinIO high-performance object storage server[1].
system	CI/CD pipelines and centralized artifact registries.	Jenkins CI, Sonatype Nexus Repository, OAuth2-Proxy authentication sidecar[1].
observability	Aggregated metrics, structured logging, and distributed tracing.	Prometheus (Server), Thanos, Loki, Grafana, Jaeger Collector, GoAlert[1].
gitops-system	Declarative state enforcement and continuous delivery.	ArgoCD Application Controller, Repository Server, and API Server[1].

1.2 Component Allocation Across the Topology

While the Admin cluster houses the centralized management plane, the operational tools interact intricately with services deployed natively inside the Development and Production clusters[1]. The architecture dictates that consumer-facing applications, alongside edge security and message-broker middleware, reside strictly in the downstream environments[1].
The downstream Development and Production clusters natively host Web Application Firewalls (WAF) to intercept public traffic, filtering out malicious payloads before they penetrate the internal network nodes[1]. Behind the WAF, Istio is deployed as a Service Mesh to enforce rigorous mutual TLS (mTLS) between microservices and manage advanced traffic splitting for

canary releases[1]. Kafka operates within these downstream clusters as the primary event-streaming message broker, running as a lightweight instance in Development and a highly available multi-broker cluster in Production[1]. Furthermore, Redocly and SwaggerUI are bundled directly beside the microservice codebases in Dev and Prod, automatically generating interactive OpenAPI documentation portals for engineering clients[1]. The Admin cluster's role is to orchestrate, monitor, and update these downstream components without hosting them directly.

2. Foundational Engine Initialization: RKE2 Configuration

Rancher Kubernetes Engine 2 (RKE2) provides an enterprise-ready, security-hardened, and compliance-focused Kubernetes distribution[1]. As the Admin cluster operates on a single-node topology for this localized deployment, default scheduling taints that protect the control plane must be explicitly disabled[1]. Without this modification, enterprise applications such as Jenkins or Nexus would be blocked from scheduling, leaving pods permanently in a pending state[1].

2.1 RKE2 Server Configuration and Network Binding

The fundamental configuration file establishes the network binding interfaces, storage directories, and access modes. By default, virtualization hypervisors may assign an internal NAT IP address (e.g., 10.0.2.15) to the primary network interface, which isolates the cluster and prevents external communication from the Dev and Prod nodes[1]. The configuration must bypass this NAT interface and bind directly to the visible host-only private subnet[1].
The deployment commences by establishing the configuration directory and defining the core parameters within the Kubernetes runtime environment. The configuration file dictates the behavior of the kubelet and the API server during the bootstrap phase[1].

Parameter	Assigned Value	Architectural Justification
write-kubeconfig-mode	"0644"	Modifies default credential permissions, allowing standard administrative users to execute kubectl commands without requiring root privilege escalation[1].
node-taint	[]	Passes an empty array to completely strip the control-plane:NoSchedule taint, permitting heavy

		DevOps workloads to schedule directly onto the master node[1].
tls-san	["192.168.56.10"]	Appends the private subnet IP to the Subject Alternative Names of the TLS certificate, ensuring cryptographic handshakes succeed when external nodes query the API[1].
node-ip	"192.168.56.10"	Forces the internal cluster routing mechanisms to advertise the host-only bridged interface, explicitly overriding hidden NAT assignments[1].

2.2 Bootstrapping the Orchestrator and Resolving Race Conditions

With the configuration parameters solidified, the RKE2 installation binary is executed, and the systemd service unit is enabled[1]. The initialization phase requires several minutes to construct the internal etcd distributed key-value store, deploy the Canal CNI overlay network, and issue cryptographic certificates[1].

During network configuration tuning, if the service attempts to restart while background processes are still locking database files, a race condition may occur within the etcd directory[1]. This manifests as a severe failure where the internal node tracker mistakenly generates a hollow folder structure instead of a valid registration file, causing the control plane to crash continuously while reporting that the server is not a member of the etcd cluster[1].

To rectify this corrupted state, administrators must perform a hard cache purge. This involves completely stopping the rke2-server daemon, executing a forced termination of all orphaned containerd-shim processes left running in the background, and forcefully removing the cached node descriptor configuration files alongside the entire /var/lib/rancher/rke2/server/db directory[1]. Upon restarting the service, RKE2 will cleanly regenerate a pristine database structure matching the newly assigned 192.168.56.10 interface[1]. Following a successful boot, the administrative credentials and binary paths are appended to the shell profile, enabling seamless remote interaction via the kubectl utility[1].

3. Storage Abstraction and Ingress Routing Infrastructure

A robust persistence layer and dynamic ingress routing fabric form the prerequisite foundation

required before deploying stateful applications and web-based dashboards[1].

3.1 Local Path Provisioning for Single-Node Topologies

Enterprise Kubernetes environments typically rely on distributed block storage systems like Ceph or Longhorn to replicate data across multiple physical disks[1]. However, deploying complex distributed storage on a single-node Admin cluster introduces unnecessary computational overhead. Instead, the architecture leverages RKE2's natively bundled rancher.io/local-path storage provisioner[1].

This automated mechanism translates PersistentVolumeClaims (PVCs) directly into localized host-path directories on the underlying virtual machine disk[1]. This provides a highly performant, low-latency disk abstraction that perfectly suits the requirements of localized databases, artifact registries, and object storage engines that do not require multi-node replication[1].

3.2 Ingress Controller Buffering Tuning

The Admin cluster utilizes the bundled NGINX Ingress Controller to route inbound HTTP and HTTPS traffic from the .nip.io wildcard domain directly to the internal cluster services[1]. However, the default NGINX configuration is optimized for standard web traffic and lacks the memory buffering capacity required to process the massive HTTP headers generated by enterprise Single Sign-On (SSO) workflows[1].

When Keycloak and OAuth2-Proxy exchange cryptographic identity tokens, the resulting JSON Web Tokens (JWTs) embedded within the browser cookies frequently exceed standard size limits. If the ingress controller is not tuned, NGINX will truncate these cookies, resulting in persistent 502 Bad Gateway loops[1]. To resolve this, all ingress manifests targeting authentication-heavy services must include specific buffering annotations[1]. The proxy buffer size is expanded to 128k, the number of proxy buffers is increased to 4, and the busy buffer size limit is extended to 256k, ensuring the successful transmission of massive OIDC payloads[1].

4. Central Identity Provider: The Identity-Core Stack

To implement true Single Sign-On (SSO) across the multi-cluster fleet, identity management must be completely decoupled from individual applications[1]. Keycloak operates as the absolute single source of truth for the entire organization[1]. All downstream platforms—including Sonatype Nexus, ArgoCD, Grafana, Rancher, and internal microservices—delegate user authentication to this central hub via OIDC or SAML protocols[1].

4.1 Ephemeral State Prevention via PostgreSQL

Keycloak requires a robust relational database schema to persist user directories, client configurations, and federated realm settings. Deploying Keycloak with an ephemeral in-memory database results in a "disappearing realm" failure scenario, where all SSO configurations vanish instantly upon a pod restart or node reboot[1]. To guarantee persistence, a dedicated PostgreSQL database is provisioned within the identity-core namespace, backed by a 10-gigabyte PVC utilizing the local-path provisioner[1].

The PostgreSQL deployment is securely configured with strict environmental variables defining

the database name, administrative user, and a secure password[1]. A dedicated internal Kubernetes Service exposes port 5432, allowing Keycloak to establish a stable JDBC connection over the internal cluster network[1].

4.2 Keycloak Initialization and Realm Configuration

The Keycloak application container is deployed alongside the database, explicitly instructed to bypass its default ephemeral storage and connect directly to the postgres-keycloak-service[1]. The deployment parameters mandate strict hostname resolution policies to be disabled (KC_HOSTNAME_STRICT="false") and proxy modes to be set to edge, accommodating the TLS termination handled by the upstream NGINX Ingress Controller[1].

Once the container reaches a running state, the service is exposed externally via the ingress controller at a dedicated subdomain. Within the administrative console, the security architecture transitions away from legacy LDAP directory synchronization[1]. While initial architectural drafts may attempt to federate Keycloak with an OpenLDAP container to share user pools, executing a "Pure Keycloak Integration" is highly recommended for streamlined, cloud-native deployments[1]. Under this paradigm, Keycloak natively stores all users, groups, and roles directly within its PostgreSQL schema, eliminating the complex synchronization logic and write-back failure loops frequently encountered with external LDAP bridges[1].

For downstream services to utilize Keycloak, dedicated OpenID Connect clients must be generated. For example, integrating the Nexus artifact registry requires the creation of a client named nexus[1]. The capability configuration must be set to Confidential to enforce client secret verification[1]. Furthermore, the valid redirect URIs must be explicitly mapped to the proxy callback endpoint (e.g., https://nexus.192.168.56.10.nip.io/oauth2/callback), and the web origins must be configured to permit Cross-Origin Resource Sharing (CORS) across the cluster domain[1].

5. S3-Compatible Object Storage: MinIO Deployment

Enterprise platform tools heavily depend on scalable, S3-compatible object storage. Loki requires S3 endpoints for long-term log chunk retention, Thanos utilizes it for historical metric downsampling and archival, and Jenkins leverages it for pipeline build artifacts[1]. Rather than introducing external dependencies and egress costs by relying on commercial cloud providers like AWS S3, the Admin cluster hosts a centralized MinIO engine inside the storage-system namespace[1].

5.1 MinIO Architecture and Resource Allocation

MinIO is a high-performance, distributed object storage server built specifically for cloud-native workloads[3]. The deployment provisions a highly performant API endpoint operating on port 9000, alongside a graphical management console exposed on port 9001[3].

To ensure data permanence, the MinIO deployment is fundamentally tied to a PersistentVolumeClaim requesting substantial disk allocation[3]. The container parameters explicitly define the root access credentials via environmental variables (MINIO_ROOT_USER and MINIO_ROOT_PASSWORD), providing administrators with absolute control over bucket

creation and policy enforcement[3].

5.2 Bucket Initialization and IAM Policy Design

Following the successful initialization of the container, administrators must navigate to the MinIO web console exposed by the ingress routing layer[3]. Within this interface, structural buckets must be pre-provisioned to support the upstream observability pipeline[12]. Specifically, buckets named loki-chunks, loki-ruler, loki-admin, and thanos-metrics are created[12].
MinIO provides fine-grained, IAM-like access control policies[10]. Rather than utilizing the root credentials for application integrations, dedicated service accounts and access keys must be generated specifically for Loki and Thanos[10]. These policies enforce a least-privilege security model, ensuring that the logging engine cannot inadvertently overwrite or delete telemetry data owned by the metrics engine[10].

6. The Platform Tooling Layer: Continuous Integration and Artifact Management

The core logic engines responsible for compiling source code into deployable images and maintaining canonical artifact registries reside in the system namespace[1]. This namespace serves as the definitive source of truth for the organization's software supply chain[1].

6.1 Continuous Integration: Jenkins Optimization

Jenkins operates as the central automation server, executing pipeline stages for compiling, testing, and formatting developer pushes[1]. Deploying Jenkins within a Kubernetes environment relies on the official Helm charts to seamlessly integrate with the cluster's native API for dynamic build-agent provisioning[1].
Because Jenkins is a highly resource-intensive Java application, placing it on the control plane requires strict resource governance[1]. Without intervention, the Java Virtual Machine (JVM) may attempt to consume the entire node's available RAM, triggering an Out-Of-Memory (OOM) panic that could destabilize the RKE2 kube-apiserver[1]. To mitigate this hazard, precise memory constraints are injected directly into the deployment configuration via the javaOpts parameter[1]. By setting constraints such as -Xms1.5g -Xmx1.5g, Jenkins is strictly capped, preventing random expansion during heavy pipeline compilations[1]. Furthermore, persistent volumes are attached to safeguard pipeline job structures, plugin configurations, and workspace data across restarts[1].

6.2 Artifact Management: Sonatype Nexus and OAuth2-Proxy Sidecar

Sonatype Nexus OSS acts as the ultimate authority for container images, Maven packages, and Helm charts[1]. All code verified by Jenkins is packaged and uploaded directly into this repository[1]. However, the open-source community edition of Nexus lacks native OpenID Connect (OIDC) capabilities, rendering it fundamentally incompatible with Keycloak[1].
To circumvent this critical limitation securely, an advanced sidecar architecture is deployed[1]. An oauth2-proxy container is injected into the exact same Kubernetes Pod as the nexus container,

forcing them to share a highly isolated local network namespace (localhost)[1]. This pattern effectively bridges the gap between Authentication (handled by Keycloak) and Authorization (handled by Nexus)[1].

6.2.1 The Remote User Token (RUT) Proxy Flow

The interaction between the ingress controller, the sidecar proxy, and the Nexus engine operates on a strict sequence of events:

1. Edge traffic targets the Nexus subdomain and is routed exclusively to the oauth2-proxy container, which listens on port 4180[1].
2. The proxy intercepts unauthenticated requests, bouncing the user's browser to the Keycloak authentication portal[1].
3. Upon successful cryptographic verification by Keycloak, the proxy extracts the identity string (e.g., admin or a developer's username) from the returned JWT payload[1].
4. The proxy injects this identity string into a custom HTTP header, specifically X-Forwarded-User, and forwards the enriched request across the internal loopback interface (127.0.0.1) directly to the Nexus engine listening on port 8081[1].
5. Nexus utilizes its native **Rut Auth Realm** capability to read the X-Forwarded-User header, blindly trusting the injected identity and logging the user in automatically without presenting a secondary credential prompt[1].

Network isolation is the paramount security directive in this design[1]. Because the Remote User Token realm explicitly trusts incoming HTTP headers, exposing the Nexus port (8081) directly to the external network would allow malicious actors to trivially spoof administrative privileges by manually injecting an X-Forwarded-User: admin header into their requests[1]. The sidecar pattern guarantees that external traffic can only enter through the proxy, which forcibly strips and overwrites any pre-existing identity headers before passing the request downstream[1].

6.2.2 Native Nexus Configuration and External Role Mapping

Because the persistent storage volume attached to Nexus initializes completely empty, the internal database contains no record of the users authenticated by Keycloak[1]. When the proxy forwards a valid user, Nexus panics upon finding zero associated roles and immediately rejects the connection with a strict 401 Unauthorized HTTP error, triggering a continuous authentication loop in the browser[1].

To resolve this, administrators must utilize a temporary port-forwarding backdoor (kubectl port-forward svc/nexus-service 8081:8081) to bypass the proxy gatekeeper and access the native UI using the randomly generated local administrator password stored within the container's /nexus-data/admin.password file[1].

Within the Nexus administrative dashboard, the security architecture is manually aligned:

1. The **Rut Auth Realm** is shifted into the active column and explicitly positioned at the absolute top of the security realm hierarchy, ensuring it intercepts identity payloads before local authentication mechanisms engage[1].
2. A system capability named Rut Auth Capability is generated, declaring X-Forwarded-User as the targeted extraction header[1].

3. An **External Role Mapping** is established to connect the incoming identity to localized permissions. The source is configured to Rut Auth, the Role ID is set identically to the Keycloak username (e.g., admin), and the native nx-admin system privilege is granted[1].
4. Finally, **Anonymous Access** is completely disabled[1].

From this point forward, Nexus fundamentally rejects any traffic lacking the proxy-injected header, securing the perimeter and enabling flawless, zero-touch Single Sign-On across the engineering team[1].

7. Centralized Continuous Delivery: ArgoCD

The Admin cluster orchestrates the deployment of verified workloads into the target Development and Production clusters utilizing GitOps paradigms powered by ArgoCD[1]. ArgoCD shifts the operational model away from manual imperative commands, enforcing desired state configurations pulled directly from Git repositories[1].

7.1 Hub-and-Spoke Deployment Architecture

ArgoCD is installed centrally within the gitops-system namespace on the Admin node[1]. The deployment encompasses a suite of microservices, including the Application Controller, Repository Server, Dex identity broker, and a Redis caching layer[1]. Rather than deploying distinct ArgoCD instances within every cluster across the infrastructure, the architecture relies on a highly efficient Hub-and-Spoke model[1].

To manage downstream environments, ArgoCD leverages North-South API server routing[1]. When the kubeadm-based Development cluster (192.168.56.11) and RKE2-based Production cluster (192.168.56.12) are initialized, their respective kubeconfig cryptographic contexts are securely imported into the Admin shell environment[1]. Using the ArgoCD Command Line Interface, the target clusters are permanently registered into ArgoCD's internal database[1]. ArgoCD securely stores these credentials and utilizes them to communicate directly with the downstream Kubernetes API servers over HTTPS (port 6443) across the host-only virtual network bridge[1]. By continuously polling upstream Git repositories, the Application Controller detects configuration drifts and autonomously pushes updated container image tags, Redocly OpenAPI definitions, or Istio routing configurations directly into the target namespaces on the external nodes[1].

8. Telemetry, Observability, and Alerting: The Telemetry Brain

To guarantee that intensive log aggregations, trace analysis, and metric queries do not consume compute resources allocated for live consumer applications, all long-term observability databases are hosted entirely within the Admin cluster's observability namespace[1].

8.1 Long-Term Metric Retention: Prometheus and Thanos

A foundational Prometheus server operates on the Admin node to scrape localized

control-plane metrics[20]. However, to achieve global visibility, it is heavily augmented by Thanos[20]. Thanos introduces a highly available metric system capable of unlimited storage capacity and seamless data deduplication[20].
Prometheus instances deployed on the downstream Development and Production nodes are configured specifically in **Agent Mode**[1]. This lightweight configuration completely disables their local Time Series Database (TSDB) storage engines[5]. Instead, these agents execute rapid scraping loops against local microservices and immediately forward the metrics back to the Admin cluster utilizing the remote_write protocol[5]. The Remote Write mechanism relies on a local Write-Ahead Log (WAL) to buffer data, ensuring that metrics are not lost during transient network disconnects across the 192.168.56.x subnet before compressing and transmitting them via HTTP POST requests[5].
The incoming telemetry stream is intercepted by the Thanos Receiver component deployed on the Admin node[5]. The Receiver passes the data to the Thanos Store Gateway and Compactor, which ultimately persist the downsampled historical metrics directly into the MinIO thanos-metrics bucket[20]. The connection to MinIO is dictated by an injected bucket.yml specification[26].

```yaml
# bucket.yml
type: s3
config:
  bucket: thanos-metrics
  endpoint: minio.storage-system.svc.cluster.local:9000
  access_key: admin_storage
  secret_key: StorageSecret99!
  insecure: true
```

8.2 Log Aggregation: Grafana Loki

Loki functions as a highly scalable log aggregation system that eschews full-text indexing in favor of metadata labels, operating similarly to Prometheus but specifically tailored for logs[27]. Logs originating from microservices in the downstream environments are collected by Promtail agents deployed as DaemonSets on the Dev and Prod worker nodes[1].
These agents format and transmit the log streams to the centralized Loki instance on the Admin cluster[27]. To manage the massive data volume without overwhelming the Admin node's ephemeral disk, Loki is configured via Helm to utilize the s3 storage driver[4]. The configuration instructs Loki to index and archive all log chunks directly into the pre-provisioned MinIO object

store[13].

8.3 Distributed Tracing & Incident Response Alerting

To provide engineering teams with end-to-end visibility into microservice latency and API bottlenecks, Jaeger is integrated into the observability suite[1]. Traces generated by application code in Dev and Prod are collected by localized OpenTelemetry sidecars and streamed via gRPC directly to the Jaeger Collector hosted in the Admin cluster[1]. Like Thanos and Loki, Jaeger is configured to utilize the MinIO S3 backend for trace persistence[14].

Grafana operates as the unified visualization layer, natively bridging the MinIO-backed Thanos, Loki, and Jaeger databases into a single pane of glass[1]. Alerting rules defined within Grafana actively monitor critical thresholds, such as elevated HTTP 500 error rates recorded by the downstream Web Application Firewalls. Upon a threshold breach, Grafana dispatches webhooks to **GoAlert**, an open-source on-call scheduling and incident response management platform deployed within the Admin cluster[1]. GoAlert, configured with its own robust PostgreSQL backend via the tokens-studio Helm chart, orchestrates automated escalations, paging the appropriate engineering teams to initiate immediate incident remediation[32].

9. Strategic Extensibility: Rancher Multi-Cluster Management

While ArgoCD handles declarative code-to-pod deployments, the overarching requirement for global fleet governance, dynamic cluster scaling, and unified visual monitoring necessitates a macro-level control plane[1]. Rancher fulfills this mandate and is deployed into the dedicated cattle-system namespace on the Admin node[1].

Once the Rancher web interface initializes, administrators gain access to a centralized cluster management dashboard[1]. Integrating the external environments is remarkably streamlined; administrators generate a distinct registration manifest from the UI and apply it via kubectl to the downstream kubeadm Dev cluster and RKE2 Prod cluster[1]. This action deploys a lightweight communication agent onto the downstream nodes, instructing them to establish a persistent websocket connection back to the Admin hub[1].

This assimilation empowers Rancher to overlay its logical "Projects" hierarchy across the physical infrastructure[1]. By integrating Rancher directly with Keycloak, administrators can map organizational SSO groups to granular RBAC policies that seamlessly span across all three independent Kubernetes clusters, delivering true enterprise-grade multi-tenant governance[1].

10. Conclusion

The rigorous initialization of the Administration cluster yields a highly resilient, single-node RKE2 platform explicitly optimized for structural governance and centralized operations. By systematically disabling protective taints, establishing explicit Host-Only Subnet routing paradigms, and segmenting traffic via North-South NGINX Ingress configurations, the ecosystem allows resource-intensive software to execute securely within highly isolated namespaces.

The architecture establishes an unbroken chain of custody for the software supply chain: developers commit code, Jenkins executes compilation pipelines, Sonatype Nexus securely warehouses the resulting artifacts via Keycloak-backed SSO authentication, and ArgoCD seamlessly orchestrates their delivery to downstream environments. Concurrently, the robust observability stack—anchored by MinIO, Thanos, Loki, and GoAlert—guarantees persistent, high-fidelity monitoring of all external workloads. The Administration control plane is now definitively prepared to govern, secure, and scale the overarching Development and Production cluster landscapes.

Works cited

1. hpe, uploaded:hpe
2. How to Configure RKE2 Server Configuration File - OneUptime, https://oneuptime.com/blog/post/2026-03-20-rke2-server-config-file/view
3. Deploying MinIO S3-Compatible Object Storage on Kubernetes and OpenShift Using YAML, https://medium.com/@madeva.tuppad/deploying-minio-s3-compatible-object-storage-on-kubernetes-and-openshift-using-yaml-b5da8f3080ea
4. Install the simple scalable Helm chart | Grafana Loki documentation, https://grafana.com/docs/loki/latest/setup/install/helm/install-scalable/
5. Prometheus Remote Write: Architecture, Examples & Troubleshooting - Groundcover, https://www.groundcover.com/learn/observability/prometheus-remote-write
6. Server Configuration Reference - RKE2, https://docs.rke2.io/reference/server_config
7. How to Configure RKE2 Server Nodes - Config - OneUptime, https://oneuptime.com/blog/post/2026-03-20-rke2-server-nodes-config/view
8. When Proxies Become the Attack Vectors in Web Architectures, https://securityboulevard.com/2026/03/when-proxies-become-the-attack-vectors-in-web-architectures/
9. Kubernetes cluster with RKE2 - Latitude.sh Docs, https://www.latitude.sh/docs/guides/kubernetes-cluster-rke2
10. Building S3-compatible Exascale object storage with MinIO - AWS in Plain English, https://aws.plainenglish.io/building-s3-compatible-exascale-object-storage-with-minio-8a8ca3fb36c4
11. Minio on Kubernetes. Object storage is useful when your... | by Aditya Joshi | Level Up Coding, https://levelup.gitconnected.com/minio-on-kubernetes-71ce34da7a19
12. Loki (v3.1.1) SimpleScalable Setup with Helm - Retention : r/grafana - Reddit, https://www.reddit.com/r/grafana/comments/1fe76ul/loki_v311_simplescalable_setup_with_helm_retention/
13. Configure storage | Grafana Loki documentation, https://grafana.com/docs/loki/latest/setup/install/helm/configure-storage/
14. Integrations & Partners Blog Posts - MinIO, https://www.min.io/category/integrations-partners?7ab4a456_page=3

15. Installing the RKE2 Server | philprime.dev, https://philprime.dev/guides/migrating-k3s-to-rke2/lesson-5.html
16. meln5674/nexus-oidc-proxy: True OpenID Connect (OIDC) integration with Nexus Repository Manager - GitHub, https://github.com/meln5674/nexus-oidc-proxy
17. mjtrangoni/oauth2-proxy-nexus3 - GitHub, https://github.com/mjtrangoni/oauth2-proxy-nexus3
18. le-garff-yoann/oauth2-proxy-nexus3 - GitHub, https://github.com/le-garff-yoann/oauth2-proxy-nexus3
19. tumbl3w33d/nexus-oauth2-proxy-plugin - GitHub, https://github.com/tumbl3w33d/nexus-oauth2-proxy-plugin
20. “THANOS” — Monitoring with Prometheus and Grafana | by Syed Saad Ahmed - Medium, https://thesaadahmed.medium.com/thanos-monitoring-with-prometheus-and-grafana-843ed231c8a6
21. Prometheus高可用项目设计与完整落地方案-腾讯云开发者社区-腾讯云, https://cloud.tencent.com/developer/article/2551547
22. values.yaml - prometheus-community/helm-charts · GitHub, https://github.com/prometheus-community/helm-charts/blob/main/charts/prometheus/values.yaml
23. Configure Prometheus Remote Write - Sysdig Docs, https://docs.sysdig.com/en/sysdig-monitor/install-prometheus-remote-write/
24. Prometheus remote write receiver | Elastic Agent, https://www.elastic.co/docs/reference/edot-collector/components/prometheusremotewritereceiver
25. Testing thanos object storage upload without waiting 2 hours - Stack Overflow, https://stackoverflow.com/questions/75845334/testing-thanos-object-storage-upload-without-waiting-2-hours
26. Thanos Store Gateway, https://thanos.io/tip/components/store.md/
27. Kubernetes with Data Engineering approach — 4 Log Collection in Kubernetes with Loki, Promtail, MinIO, and Grafana | by AHMET FURKAN DEMIR | Medium, https://medium.com/@ahmetfurkandemir/log-collection-in-kubernetes-with-loki-promtail-minio-and-grafana-39b6e3b6b799
28. How to Deploy Loki on Kubernetes with Helm - OneUptime, https://oneuptime.com/blog/post/2026-01-21-loki-kubernetes-helm/view
29. Deployment - Jaeger, https://www.jaegertracing.io/docs/1.76/deployment/
30. goalert 0.0.5 - Tokens Studio - Artifact Hub, https://artifacthub.io/packages/helm/tokens-studio/goalert
31. geico repositories · GitHub, https://github.com/orgs/geico/repositories
32. underfisk/awesome-go-1 - GitHub, https://github.com/underfisk/awesome-go-1
33. goalert 0.0.4 - Tokens Studio - Artifact Hub, https://artifacthub.io/packages/helm/tokens-studio/goalert/0.0.4